Actividad: productor–consumidor y filósofos comensales

Objetivo

Implementar dos problemas de sincronización con monitores en Java y determinar si las soluciones presentan:

- Problemas de exclusión mutua.
- Interbloqueo.
- Inanición.

Parte 1. Productor–consumidor

Implementen un búfer circular con capacidad para cinco elementos.

El programa debe utilizar:

- Dos productores y dos consumidores.
- Métodos deposit() y fetch() con synchronized.
- wait() y; notify( ) o notifyAll().
- Condiciones verificadas mediante while.

Cada productor generará 20 elementos y cada consumidor retirará 20.

Comprueben que:

- No se deposite cuando el búfer esté lleno.
- No se retire cuando esté vacío.
- Ningún elemento se pierda o consuma dos veces.
- Todos los hilos terminen.

Parte 2. Filósofos comensales

Implementen cinco filósofos mediante hilos y un monitor que controle sus estados:

PENSANDO → HAMBRIENTO → COMIENDO

El monitor debe incluir:

- Un arreglo con el estado de los filósofos.
- Un método para solicitar permiso para comer.

- Un método para terminar de comer.
- Un método test(i) para revisar a los vecinos.
- Uso de synchronized, wait() y; notify( ) o notifyAll().

Cada filósofo debe intentar comer 20 veces. Registren cuántas veces comió cada uno.

Pruebas

Ejecuten ambos programas al menos diez veces, utilizando tiempos aleatorios para producir, consumir, pensar y comer.

Ejecución	**¿Terminó?**	**¿Hubo acceso simultáneo incorrecto?**	**¿Algún hilo dejó de avanzar?**
1			
…			
10			

Análisis

Para cada solución respondan:

1. ¿Se garantiza la exclusión mutua?
2. ¿Puede ocurrir un interbloqueo?
3. ¿Puede ocurrir inanición?
4. ¿Qué evidencia respalda sus respuestas?
5. ¿Qué modificación propondrían si detectaran algún problema?

Entregables

- Código de los dos programas.
- Evidencia de las ejecuciones.
- Tabla de resultados.
- Análisis y conclusiones.